Node.js — Complete Knowledge Refresher (Full Stack Engineer)

> Everything you need to know about Node.js — runtime internals, event loop, async patterns, Express/NestJS, security, performance, testing, and production operations. Interview-calibrated for 7+ YOE.

Table of Contents

1. What is Node.js & Why It Exists
2. Architecture — V8, libuv, Event Loop
3. The Event Loop — Phase by Phase
4. Modules — CommonJS vs ES Modules
5. Async Patterns — Callbacks, Promises, Async/Await
6. Streams & Buffers
7. Error Handling — The Right Way
8. Express.js — The De Facto Framework
9. NestJS — Enterprise Node.js
10. REST API Design Patterns
11. Authentication & Authorization
12. Database Access — ORM, Query Builders, Drivers
13. Security — OWASP Top 10 for Node.js
14. Performance & Scaling
15. Testing — Unit, Integration, E2E
16. TypeScript with Node.js
17. Node.js Internals — Memory, GC, Diagnostics
18. Microservices Patterns in Node.js
19. Node.js + Docker + K8s (Production)
20. Common Interview Questions & Answers

---

1. What is Node.js & Why It Exists

```
Node.js is a JavaScript runtime built on Chrome's V8 engine.
It lets you run JavaScript OUTSIDE the browser — on servers, CLIs, anywhere.

Created by Ryan Dahl in 2009 to solve a specific problem:
  Apache httpd handled each request with a THREAD.
  10,000 concurrent connections = 10,000 threads = 10,000 × 1MB stack = 10GB RAM.
  Threads are expensive. Context switching kills throughput.

Node.js solution: SINGLE THREAD + EVENT LOOP + NON-BLOCKING I/O.
  10,000 concurrent connections = 1 thread + events = ~100MB RAM.
  I/O operations (disk, network, DB) are delegated to the OS/libuv thread pool.
  When I/O completes, a callback fires on the event loop.

What Node.js IS:
  ✅ Great for I/O-heavy workloads (APIs, real-time, streaming)
  ✅ Non-blocking by design
  ✅ Huge ecosystem (npm — millions of packages)
  ✅ Same language (JS/TS) on frontend and backend

What Node.js is NOT:
  ❌ Not ideal for CPU-heavy computation (image processing, ML, heavy math)
      → Blocks the event loop → all requests stall
      → Solution: worker_threads, or offload to separate service
  ❌ Not multi-threaded by default (but you can use cluster/worker_threads)
```

2. Architecture — V8, libuv, Event Loop

```
┌──────────────────────── YOUR CODE (JavaScript / TypeScript)
────────────────────────┐
│
│
│  const server = http.createServer((req, res) => {
│
│    db.query('SELECT ...', (err, rows) => { res.json(rows); });
│
│  });
│
│
│
```

```
└──────────────────────────────┴──────────────────────────────────────────────────────────────────────────────
───┘
                               │
                     ┌─────────▼──────────┐
                     │    NODE.JS RUNTIME   │
                     │  ┌────────────────┐  │
                     │  │  Node.js APIs  │  │ ← http, fs, crypto, path, os, etc.
                     │  │  (JS bindings) │  │
                     │  └────────┬───────┘  │
                     │           │           │
                     │  ┌────────▼────────┐  │
                     │  │   V8 ENGINE     │  │ ← compiles JS → machine code (JIT)
                     │  │  (Google C++)   │  │   heap management, garbage
collection
                     │  └────────┬───────┘  │
                     │           │           │
                     │  ┌────────▼────────┐  │
                     │  │     libuv       │  │ ← event loop, async I/O, thread
pool
                     │  │   (C library)   │  │   cross-platform abstraction
                     │  └────────┬───────┘  │
                     └───────────┼──────────┘
                                 │
                     ┌───────────▼──────────┐
                     │   OPERATING SYSTEM   │
                     │  epoll (Linux)       │
                     │  kqueue (macOS)      │
                     │  IOCP (Windows)      │
                     └──────────────────────┘
```

2.1 V8 Engine

```
What it does:
  1. Parses JavaScript into an AST (Abstract Syntax Tree)
  2. Compiles to bytecode (Ignition interpreter)
  3. Hot functions → JIT compiled to optimized machine code (TurboFan)
  4. Manages heap memory and garbage collection

Key concepts:
  Hidden Classes: V8 creates hidden classes for objects to speed up property
access.
    → Consistent object shapes = fast code. Adding properties dynamically = slow.

  Inline Caching: V8 caches property lookup locations.
```

```
    → Monomorphic calls (same type every time) are fastest.

  Garbage Collection:
    Young Generation (Scavenger): small, frequent, fast (minor GC)
    Old Generation (Mark-Sweep/Compact): large, less frequent (major GC)

  Heap limit: ~1.5GB by default on 64-bit systems.
    Override: node --max-old-space-size=4096 app.js  (4GB)
```

2.2 libuv

```
Cross-platform async I/O library (written in C).
THE engine behind Node.js's non-blocking model.

What libuv provides:
  1. Event loop implementation
  2. Thread pool (default 4 threads, max 1024)
  3. Async TCP/UDP sockets
  4. Async DNS resolution
  5. File system operations
  6. Child processes
  7. Signal handling
  8. Timers

Thread pool (UV_THREADPOOL_SIZE):
  Used for operations that CAN'T be done async at the OS level:
    - fs operations (read, write, stat)
    - DNS lookup (dns.lookup, NOT dns.resolve)
    - crypto (pbkdf2, randomBytes, scrypt)
    - zlib (compression)

  Default: 4 threads
  Tune: UV_THREADPOOL_SIZE=16 node app.js
  Max: 1024

  Network I/O does NOT use the thread pool:
    - Uses OS-level async (epoll/kqueue/IOCP) via libuv
    - This is why Node handles thousands of connections with one thread
```

---

3. The Event Loop — Phase by Phase

This is the #1 Node.js interview topic. Know it cold.

```
   ┌───────────────────────────┐
┌─>│         timers            │  ← setTimeout, setInterval callbacks
│  └─────────────┬─────────────┘
│  ┌─────────────▼─────────────┐
│  │     pending callbacks     │  ← I/O callbacks deferred from previous loop
│  └─────────────┬─────────────┘
│  ┌─────────────▼─────────────┐
│  │       idle, prepare       │  ← internal use only
│  └─────────────┬─────────────┘
│  ┌─────────────▼─────────────┐
│  │          poll             │  ← retrieve new I/O events, execute I/O
callbacks
│  └─────────────┬─────────────┘     (most of your code runs here)
│  ┌─────────────▼─────────────┐
│  │         check             │  ← setImmediate callbacks
│  └─────────────┬─────────────┘
│  ┌─────────────▼─────────────┐
│  │    close callbacks        │  ← socket.on('close'), cleanup
│  └─────────────┬─────────────┘
│                │
│  ┌─────────────▼─────────────┐
│  │  process.nextTick queue   │  ← runs between EVERY phase transition
│  │  + microtask queue        │    (Promise .then / .catch / .finally)
│  └─────────────┬─────────────┘
└────────────────┘
```

Phase Details

```
1. TIMERS
   Executes callbacks from setTimeout() and setInterval().
   Timers are NOT precise — they fire AFTER the delay, not exactly AT it.
   If poll phase takes 100ms, a 50ms timer fires at ~100ms.

2. PENDING CALLBACKS
   Executes callbacks for certain system operations (TCP errors, etc.).
   Rarely relevant in application code.

3. POLL
   THE most important phase. Two jobs:
```

```
     a) Calculate how long to block waiting for I/O
     b) Process events in the poll queue

   If poll queue is not empty → execute all callbacks (synchronously, in order)
   If poll queue is empty:
     → If setImmediate() is scheduled → go to CHECK phase
     → If timers are due → wrap back to TIMERS phase
     → Otherwise → block here waiting for new I/O events

4. CHECK
   Executes setImmediate() callbacks.
   Always runs after poll phase completes.

5. CLOSE CALLBACKS
   socket.destroy() → socket.on('close', callback) fires here.

BETWEEN EVERY PHASE:
   process.nextTick() queue is drained (FIFO)
   Then microtask queue is drained (Promise.then callbacks)
   nextTick has HIGHER priority than Promises.
```

The Classic Interview Puzzle

```
console.log('1');                        // sync → immediate

setTimeout(() => console.log('2'), 0);     // timers phase (next loop)

setImmediate(() => console.log('3'));       // check phase

Promise.resolve().then(() => console.log('4'));  // microtask

process.nextTick(() => console.log('5'));   // nextTick queue

console.log('6');                        // sync → immediate

// OUTPUT:
// 1
// 6        ← sync code finishes first
// 5        ← nextTick (highest priority microtask)
// 4        ← Promise microtask
// 2 or 3   ← timer vs immediate (order varies in main module)
// 3 or 2   ← timer vs immediate (order varies in main module)

// INSIDE an I/O callback, setImmediate ALWAYS fires before setTimeout:
```

```
const fs = require('fs');
fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate'));
});
// OUTPUT: immediate, timeout (guaranteed)
// Because: after poll phase, check (setImmediate) runs before wrapping to timers
```

Event Loop — Why It Matters

```
BLOCKING THE EVENT LOOP = DISASTER

  // BAD: blocks event loop for 5 seconds
  function fibonacci(n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
  }
  app.get('/fib', (req, res) => {
    const result = fibonacci(45);  // blocks ~5s
    res.json({ result });
    // EVERY other request waits 5s. Server is frozen.
  });

  // GOOD: offload to worker thread
  const { Worker } = require('worker_threads');
  app.get('/fib', (req, res) => {
    const worker = new Worker('./fib-worker.js', { workerData: 45 });
    worker.on('message', (result) => res.json({ result }));
    worker.on('error', (err) => res.status(500).json({ error: err.message }));
  });

Event loop health check:
  Monitor event loop lag. If lag > 100ms, something is blocking.
  Libraries: clinic.js, prom-client (event loop lag metric)
```

---

4. Modules — CommonJS vs ES Modules

```
┌──────────────────────┬──────────────────────────┬────────────────────┐
│                      │ CommonJS (CJS)           │ ES Modules (ESM)   │
├──────────────────────┼──────────────────────────┼────────────────────┤
```

```
│ Syntax                  │ require() / module.exports │ import / export │
│ Loading                 │ Synchronous                │ Asynchronous    │
│ When loaded             │ Runtime (dynamic)          │ Parse time (static)│
│ File extension          │ .js (default)              │ .mjs or "type":"module" │
│ Top-level await         │ ❌ No                      │ ✅ Yes          │
│ Tree-shakeable          │ ❌ No                      │ ✅ Yes (static imports) │
│ this context            │ module.exports             │ undefined       │
│ __dirname / __filename│ ✅ Available               │ ❌ Use import.meta.url │
│ Circular deps           │ Partial exports returned│ Live bindings (references) │
│ Node.js default         │ ✅ Yes                     │ Opt-in          │
└────────────────────────┴────────────────────────────┴─────────────────┘


// CommonJS
const express = require('express');
module.exports = { myFunction };

// ES Modules
import express from 'express';
export const myFunction = () => {};
export default myFunction;

// Dynamic import (works in both systems)
const module = await import('./myModule.js');

Module resolution order (require('myModule')):
  1. Core module? (fs, http, path) → return built-in
  2. Starts with ./ or ../ or / → resolve as file/directory
  3. Look in node_modules/ (walk up directory tree)
  4. Global modules (NODE_PATH)

Caching: require() caches modules by resolved filename.
  require('./config') returns the SAME object every time.
  → Singleton pattern is built into CommonJS.
```

---

5. Async Patterns — Callbacks, Promises, Async/Await

5.1 Callbacks (Legacy)

```
// Error-first callback pattern (Node convention)
fs.readFile('/etc/passwd', 'utf8', (err, data) => {
```

```
  if (err) {
    console.error('Failed:', err);
    return;
  }
  console.log(data);
});

// The problem: Callback Hell
getUser(id, (err, user) => {
  getOrders(user.id, (err, orders) => {
    getOrderDetails(orders[0].id, (err, details) => {
      sendEmail(user.email, details, (err, result) => {
        // 4 levels deep, unreadable, error handling nightmare
      });
    });
  });
});
```

5.2 Promises

```
// Promise = a container for a future value
// States: pending → fulfilled or rejected (settled)

function getUser(id) {
  return new Promise((resolve, reject) => {
    db.query('SELECT * FROM users WHERE id = ?', [id], (err, rows) => {
      if (err) reject(err);
      else resolve(rows[0]);
    });
  });
}

// Chaining (solves callback hell)
getUser(1)
  .then(user => getOrders(user.id))
  .then(orders => getOrderDetails(orders[0].id))
  .then(details => sendEmail(details))
  .catch(err => console.error(err));  // single error handler for entire chain

// Concurrent execution
const [users, orders, inventory] = await Promise.all([
  getUsers(),
  getOrders(),
  getInventory()
```

```
]);
// All three run in PARALLEL, resolves when ALL complete

// Promise.allSettled — don't fail fast, get all results
const results = await Promise.allSettled([api1(), api2(), api3()]);
// results: [{status: 'fulfilled', value: ...}, {status: 'rejected', reason:
...}]

// Promise.race — first to settle wins
const result = await Promise.race([
  fetch(primaryUrl),
  new Promise((_, reject) => setTimeout(() => reject(new Error('Timeout')),
5000))
]);

// Promise.any — first to FULFILL wins (ignores rejections)
const fastest = await Promise.any([mirror1(), mirror2(), mirror3()]);
```

5.3 Async/Await (Modern Standard)

```
// Syntactic sugar over Promises. Makes async code read like sync.

async function processOrder(orderId) {
  try {
    const order = await getOrder(orderId);        // waits, doesn't block event
loop
    const user = await getUser(order.userId);
    const payment = await chargeCard(user, order);
    await sendConfirmation(user.email, payment);
    return { success: true };
  } catch (err) {
    // any await rejection lands here
    logger.error('Order processing failed', { orderId, error: err.message });
    throw err;  // re-throw or handle
  }
}

// PARALLEL with async/await (common mistake: running sequentially)

// ❌ SEQUENTIAL — each waits for the previous
const users = await getUsers();           // 200ms
const orders = await getOrders();         // 300ms
const inventory = await getInventory();   // 150ms
// Total: 650ms
```

```
// ✅ PARALLEL — all start simultaneously
const [users, orders, inventory] = await Promise.all([
  getUsers(),           // 200ms
  getOrders(),          // 300ms
  getInventory()        // 150ms
]);
// Total: 300ms (slowest one)

// Top-level await (ES Modules only)
const config = await loadConfig();
export default config;
```

---

6. Streams & Buffers

6.1 Buffers

```
// Buffer = raw binary data (like byte[] in Java)
// Used for file I/O, network protocols, binary data

const buf = Buffer.from('Hello', 'utf8');  // <Buffer 48 65 6c 6c 6f>
const buf2 = Buffer.alloc(10);             // 10 zero-filled bytes
const buf3 = Buffer.allocUnsafe(10);       // 10 uninitialized bytes (faster, but
may contain old data)

buf.toString('utf8');    // 'Hello'
buf.toString('base64');  // 'SGVsbG8='
buf.length;              // 5 (bytes, not characters)

// Buffers are NOT resizable. If you need more space, create a new buffer.
```

6.2 Streams

```
Streams process data PIECE BY PIECE instead of loading everything into memory.

Why streams matter:
  Without streams: read 2GB file → 2GB in memory → process → write
  With streams:    read 64KB chunk → process → write → read next chunk
  Memory usage: 2GB vs ~64KB
```

```
Four types:
  Readable:    source of data (fs.createReadStream, http request)
  Writable:    destination (fs.createWriteStream, http response)
  Duplex:      both readable & writable (TCP socket)
  Transform:   duplex that modifies data (zlib.createGzip)
```

```
// Stream a large file (memory-efficient)
const fs = require('fs');

// ❌ BAD: loads entire file into memory
app.get('/download', (req, res) => {
  const data = fs.readFileSync('/large-file.csv');  // 2GB in memory!
  res.send(data);
});

// ✅ GOOD: streams in chunks
app.get('/download', (req, res) => {
  const stream = fs.createReadStream('/large-file.csv');
  stream.pipe(res);  // chunks flow from file → response, ~64KB at a time
});

// Transform stream: compress on the fly
const zlib = require('zlib');
fs.createReadStream('input.log')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('input.log.gz'));

// Stream events
stream.on('data', (chunk) => { /* process chunk */ });
stream.on('end', () => { /* no more data */ });
stream.on('error', (err) => { /* handle error */ });

// Backpressure: if writable is slower than readable,
// pipe() automatically pauses the readable stream.
// This prevents memory overflow.

// Modern: pipeline() with error handling
const { pipeline } = require('stream/promises');
await pipeline(
  fs.createReadStream('input.csv'),
  transformStream,
  fs.createWriteStream('output.csv')
);
// properly handles errors and cleanup
```

7. Error Handling — The Right Way

```
// ─── SYNC ERRORS ───
try {
  JSON.parse(invalidJson);
} catch (err) {
  // caught
}

// ─── ASYNC ERRORS (Promises) ───

// ✅ try/catch with async/await
async function handler(req, res) {
  try {
    const data = await fetchData();
    res.json(data);
  } catch (err) {
    res.status(500).json({ error: 'Internal server error' });
  }
}

// ✅ .catch() on Promises
fetchData()
  .then(data => process(data))
  .catch(err => logger.error(err));

// ─── UNHANDLED REJECTIONS (catch-all) ───
process.on('unhandledRejection', (reason, promise) => {
  logger.error('Unhandled Rejection:', reason);
  // In Node 15+, unhandled rejections CRASH the process by default
  // Always handle your promises!
});

// ─── UNCAUGHT EXCEPTIONS (catch-all) ───
process.on('uncaughtException', (err) => {
  logger.error('Uncaught Exception:', err);
  // Process is in an UNKNOWN state. Log and EXIT.
  process.exit(1);
  // Let K8s/PM2 restart the process
});

// ─── OPERATIONAL vs PROGRAMMER ERRORS ───
```

```
// Operational: expected failures (DB down, timeout, bad input)
//   → Handle gracefully (retry, return 400/503)
// Programmer: bugs (TypeError, null reference)
//   → Crash and restart (fix the bug)

// ─── CUSTOM ERROR CLASSES ───
class AppError extends Error {
  constructor(message, statusCode, isOperational = true) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = isOperational;
    Error.captureStackTrace(this, this.constructor);
  }
}

class NotFoundError extends AppError {
  constructor(resource) {
    super(`${resource} not found`, 404);
  }
}

class ValidationError extends AppError {
  constructor(message) {
    super(message, 400);
  }
}
```

---

8. Express.js — The De Facto Framework

8.1 Core Concepts

```
const express = require('express');
const app = express();

// ─── MIDDLEWARE ───
// Functions with (req, res, next) that execute in ORDER of registration.

// Built-in middleware
app.use(express.json());                  // parse JSON bodies
app.use(express.urlencoded({ extended: true }));
app.use(express.static('public'));        // serve static files
```

```
// Custom middleware
app.use((req, res, next) => {
  req.requestTime = Date.now();
  console.log(`${req.method} ${req.url}`);
  next();  // MUST call next() or send response, otherwise request hangs
});

// Error-handling middleware (4 parameters)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.statusCode || 500).json({
    error: err.message || 'Internal server error'
  });
});
```

8.2 Routing

```
const router = express.Router();

// RESTful routes
router.get('/users',          listUsers);       // GET    /api/users
router.get('/users/:id',      getUser);         // GET    /api/users/123
router.post('/users',         createUser);      // POST   /api/users
router.put('/users/:id',      updateUser);      // PUT    /api/users/123
router.patch('/users/:id',    patchUser);       // PATCH  /api/users/123
router.delete('/users/:id',   deleteUser);      // DELETE /api/users/123

app.use('/api', router);

// Route parameters
router.get('/users/:userId/orders/:orderId', (req, res) => {
  const { userId, orderId } = req.params;
});

// Query parameters: GET /search?q=node&page=2
router.get('/search', (req, res) => {
  const { q, page } = req.query;
});
```

8.3 Middleware Execution Order

```
Request arrives
    ↓
  app.use(cors())                    ← 1. CORS headers
    ↓
  app.use(helmet())                  ← 2. Security headers
    ↓
  app.use(express.json())            ← 3. Parse body
    ↓
  app.use(rateLimiter)               ← 4. Rate limiting
    ↓
  app.use(requestLogger)             ← 5. Log request
    ↓
  router.get('/users', authMiddleware, validate, handler)
    ↓                    ↓              ↓         ↓
    6. Route match    7. Auth check  8. Validate  9. Handler
    ↓
  app.use(errorHandler)              ← 10. Error middleware (if any threw)
    ↓
  Response sent

Key rules:
  - Order matters. Middleware runs top→down.
  - next() passes to the next middleware.
  - next(err) jumps to error-handling middleware.
  - If you send a response, the chain stops (don't call next after res.send).
```

8.4 Express Best Practices

```
// ─── Project Structure (feature-based) ───
src/
  modules/
    users/
      users.controller.js
      users.service.js
      users.route.js
      users.validation.js
      users.test.js
    orders/
      ...
  middleware/
    auth.js
```

```
    errorHandler.js
    rateLimiter.js
    validate.js
  config/
    index.js
    database.js
  utils/
    logger.js
    AppError.js
  app.js            ← express setup, middleware registration
  server.js         ← app.listen() entry point

// ─── Async Error Wrapper ───
// Avoids try/catch in every handler

const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

router.get('/users', asyncHandler(async (req, res) => {
  const users = await userService.findAll();
  res.json(users);
  // if findAll throws, asyncHandler catches it → error middleware
}));

// ─── Separation of Concerns ───
// Controller: handles HTTP (req/res)
// Service: business logic (testable without Express)
// Repository/Model: database access

// Controller
const getUser = asyncHandler(async (req, res) => {
  const user = await userService.findById(req.params.id);
  if (!user) throw new NotFoundError('User');
  res.json(user);
});

// Service
class UserService {
  async findById(id) {
    const user = await User.findByPk(id);
    return user;
  }
}
```

---

9. NestJS — Enterprise Node.js

```
NestJS = Angular-inspired Node.js framework (TypeScript-first)
Built on top of Express (or Fastify).
Provides structure, DI, modules — what Express lacks.

If you know Spring Boot (Java), NestJS maps almost 1:1:
  Spring @Controller    → Nest @Controller
  Spring @Service       → Nest @Injectable
  Spring @Autowired     → Nest constructor injection
  Spring @Module        → Nest @Module
  Spring @Guard         → Nest @UseGuards
  Spring @RequestBody   → Nest @Body
  Spring Interceptor    → Nest @UseInterceptors
```

```
// ─── Module ───
@Module({
  imports: [TypeOrmModule.forFeature([User])],
  controllers: [UsersController],
  providers: [UsersService],
  exports: [UsersService],
})
export class UsersModule {}

// ─── Controller ───
@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Get()
  findAll(): Promise<User[]> {
    return this.usersService.findAll();
  }

  @Get(':id')
  findOne(@Param('id', ParseIntPipe) id: number): Promise<User> {
    return this.usersService.findOne(id);
  }

  @Post()
  @UsePipes(new ValidationPipe())
  create(@Body() dto: CreateUserDto): Promise<User> {
    return this.usersService.create(dto);
```

```
  }
}

// ─── Service ───
@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private readonly userRepo: Repository<User>,
  ) {}

  findAll(): Promise<User[]> {
    return this.userRepo.find();
  }
}

// ─── DTO with Validation ───
export class CreateUserDto {
  @IsString()
  @IsNotEmpty()
  name: string;

  @IsEmail()
  email: string;

  @IsString()
  @MinLength(8)
  password: string;
}
```

---

10. REST API Design Patterns

```
─── URL CONVENTIONS ───
  GET    /api/v1/users              → list users
  GET    /api/v1/users/123          → get user 123
  POST   /api/v1/users              → create user
  PUT    /api/v1/users/123          → replace user 123
  PATCH  /api/v1/users/123          → partial update user 123
  DELETE /api/v1/users/123          → delete user 123
  GET    /api/v1/users/123/orders   → orders for user 123

Rules:
```

```
  - Nouns, not verbs: /users not /getUsers
  - Plural: /users not /user
  - Lowercase, hyphens: /order-items not /orderItems
  - Version in URL: /api/v1/ or via header Accept: application/vnd.api.v1+json

──── PAGINATION ────
  GET /api/v1/users?page=2&limit=20
  Response:
  {
    "data": [...],
    "meta": {
      "total": 150,
      "page": 2,
      "limit": 20,
      "totalPages": 8
    }
  }

  Cursor-based (better for large datasets):
  GET /api/v1/users?cursor=abc123&limit=20

──── FILTERING, SORTING, SEARCH ────
  GET /api/v1/users?role=admin&status=active     ← filter
  GET /api/v1/users?sort=-createdAt,name          ← sort (- = desc)
  GET /api/v1/users?search=john                  ← full-text search

──── HTTP STATUS CODES ────
  200 OK               → success
  201 Created           → resource created (POST)
  204 No Content       → success, no body (DELETE)
  400 Bad Request      → validation error
  401 Unauthorized     → not authenticated
  403 Forbidden        → authenticated but not authorized
  404 Not Found        → resource doesn't exist
  409 Conflict         → duplicate, optimistic lock failure
  422 Unprocessable    → semantically invalid
  429 Too Many Requests→ rate limited
  500 Internal Error   → server bug
  503 Service Unavailable → overloaded / maintenance

──── CONSISTENT ERROR RESPONSE ────
  {
    "status": "error",
    "statusCode": 400,
    "message": "Validation failed",
```

```
    "errors": [
      { "field": "email", "message": "Invalid email format" },
      { "field": "name", "message": "Name is required" }
    ]
  }
```

11. Authentication & Authorization

11.1 JWT (JSON Web Tokens)

```
Structure: header.payload.signature (base64url encoded)

  Header:  { "alg": "HS256", "typ": "JWT" }
  Payload: { "sub": "user123", "role": "admin", "iat": 1711839600, "exp":
1711843200 }
  Signature: HMAC-SHA256(base64(header) + "." + base64(payload), secret)

Flow:
  1. Client: POST /auth/login { email, password }
  2. Server: validate credentials → generate JWT → return token
  3. Client: stores token (httpOnly cookie or memory — NOT localStorage)
  4. Client: Authorization: Bearer <token> on every request
  5. Server: verify signature, check expiry, extract user info

Token types:
  Access Token:  short-lived (15 min), used to access APIs
  Refresh Token: long-lived (7 days), stored securely, used to get new access
tokens
```

```
const jwt = require('jsonwebtoken');

// Generate
function generateTokens(user) {
  const accessToken = jwt.sign(
    { sub: user.id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: '15m' }
  );
  const refreshToken = jwt.sign(
    { sub: user.id },
    process.env.JWT_REFRESH_SECRET,
```

```
    { expiresIn: '7d' }
  );
  return { accessToken, refreshToken };
}

// Verify middleware
function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }

  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

// Authorization middleware
function authorize(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }
    next();
  };
}

// Usage
router.get('/admin/users', authenticate, authorize('admin'), getUsers);
```

11.2 OAuth 2.0 / OIDC

```
OAuth 2.0: Authorization framework (delegated access)
OIDC: Authentication layer on top of OAuth (user identity)

Common flow (Authorization Code + PKCE — for SPAs and mobile):
  1. User clicks "Login with Google"
  2. App redirects to Google's authorization endpoint
  3. User authenticates with Google, consents to permissions
```

```
  4. Google redirects back with an authorization CODE
  5. App's backend exchanges code for tokens (access + id + refresh)
  6. App uses id token to identify user, access token to call Google APIs

Libraries:
  passport.js (Express) — strategies for Google, GitHub, etc.
  @nestjs/passport (NestJS wrapper)
```

11.3 Session vs Token

```
┌──────────────────┬──────────────────┬──────────────────────┐
│                  │ Session-Based    │ Token-Based (JWT)    │
├──────────────────┼──────────────────┼──────────────────────┤
│ State            │ Stateful (server)│ Stateless            │
│ Storage          │ Server memory/DB │ Client-side          │
│ Scaling          │ Needs sticky sess│ Any server can verify│
│                  │ or shared store  │                      │
│ Revocation       │ Easy (delete sess)│ Hard (need blocklist)│
│ Size             │ Small cookie ID  │ Larger (payload)     │
│ Mobile-friendly  │ No (cookies awkward)│ Yes                 │
│ K8s friendly     │ Need Redis for   │ ✅ Stateless, any Pod│
│                  │ shared sessions  │                      │
└──────────────────┴──────────────────┴──────────────────────┘
```

12. Database Access — ORM, Query Builders, Drivers

```
─── RAW DRIVERS ───
  pg (PostgreSQL):  const { Client } = require('pg');
  mysql2:           const mysql = require('mysql2/promise');
  mongodb:          const { MongoClient } = require('mongodb');
  redis (ioredis):  const Redis = require('ioredis');

Use when: maximum performance, complex queries, full control.

─── QUERY BUILDERS ───
  Knex.js: SQL query builder + migrations
    await knex('users').where({ id: 1 }).first();
    await knex('users').insert({ name: 'John', email: 'j@j.com' });
```

```
Use when: you want SQL control without raw strings.

─── ORMs ───
  Prisma:     modern, type-safe (TypeScript), schema-first, excellent DX
  Sequelize:  mature, model-based, popular with Express
  TypeORM:    decorator-based, similar to JPA/Hibernate, popular with NestJS
  Drizzle:    lightweight, SQL-like TS ORM (newest)

Comparison:
  ┌──────────┬──────────────┬──────────────┬──────────────┐
  │          │ Prisma       │ TypeORM      │ Sequelize    │
  ├──────────┼──────────────┼──────────────┼──────────────┤
  │ TypeScript│ ✅ Native    │ ✅ Decorators│ ⚠️  Plugin    │
  │ Schema   │ .prisma file │ Entity classes│ Model defs   │
  │ Migrations│ Built-in     │ Built-in     │ CLI tool     │
  │ Relations│ Implicit     │ Decorators   │ Methods      │
  │ Raw SQL  │ $queryRaw    │ query()      │ query()      │
  │ Learning │ Easy         │ Moderate     │ Moderate     │
  │ Best with│ Any          │ NestJS       │ Express      │
  └──────────┴──────────────┴──────────────┴──────────────┘
```

```
// ─── Prisma Example ───

// schema.prisma
// model User {
//   id        Int      @id @default(autoincrement())
//   email     String   @unique
//   name      String
//   orders    Order[]
//   createdAt DateTime @default(now())
// }

const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

// CRUD
const user = await prisma.user.create({
  data: { name: 'John', email: 'john@email.com' }
});

const users = await prisma.user.findMany({
  where: { name: { contains: 'John' } },
  include: { orders: true },  // eager load relations
  orderBy: { createdAt: 'desc' },
  take: 10,
```

```
  skip: 0,
});

// Transaction
const [order, payment] = await prisma.$transaction([
  prisma.order.create({ data: { userId: 1, total: 99.99 } }),
  prisma.payment.create({ data: { orderId: 1, amount: 99.99 } }),
]);
```

Connection Pooling

```
CRITICAL for production.
Without pooling: every request opens a new DB connection (TCP handshake + auth).
With pooling: reuse connections from a pool.

pg pool:
  const pool = new Pool({
    host: process.env.DB_HOST,
    max: 20,                    // max connections in pool
    idleTimeoutMillis: 30000,   // close idle connections after 30s
    connectionTimeoutMillis: 2000, // fail if can't get connection in 2s
  });

Prisma: connection pool is built-in
  DATABASE_URL="postgresql://user:pass@host:5432/db?connection_limit=20"

Sizing rule:
  pool_size = (number_of_cores × 2) + effective_spindle_count
  For most apps: 20-50 connections per Pod
  Total across Pods must not exceed DB max_connections
```

---

13. Security — OWASP Top 10 for Node.js

```
1. INJECTION (SQL / NoSQL / Command)
   ──────────────────────────────

   ❌ BAD: Raw string interpolation
     db.query(`SELECT * FROM users WHERE id = '${req.params.id}'`);

   ✅ FIX: Parameterized queries (ALWAYS)
     db.query('SELECT * FROM users WHERE id = $1', [req.params.id]);
```

```
   ❌ BAD: NoSQL injection (MongoDB)
     User.find({ email: req.body.email, password: req.body.password });
     // Attacker sends: { "password": { "$gt": "" } }  → always true

   ✅ FIX: Validate input types, use mongoose schema validation

   ❌ BAD: Command injection
     exec(`ping ${req.query.host}`);
     // Attacker sends: host=; rm -rf /

   ✅ FIX: Use execFile() with argument arrays, never shell interpolation
     execFile('ping', ['-c', '4', validatedHost]);

2. BROKEN AUTHENTICATION
   ────────────────────────
   - Hash passwords with bcrypt (cost factor 12+)
       const hash = await bcrypt.hash(password, 12);
   - Rate limit login attempts (express-rate-limit)
   - Account lockout after N failures
   - JWT: short-lived access tokens, httpOnly cookies
   - Never store tokens in localStorage (XSS-accessible)
   - Use constant-time comparison for token/password verification

3. XSS (Cross-Site Scripting)
   ──────────────────────────────
   - Escape all user output (React does this by default, raw HTML doesn't)
   - Set Content-Security-Policy headers
   - Use helmet.js for security headers
   - Sanitize HTML input: const clean = DOMPurify.sanitize(userInput);
   - httpOnly cookies prevent JS access to session tokens

4. CSRF (Cross-Site Request Forgery)
   ────────────────────────────────────
   - Use CSRF tokens for state-changing operations (csurf middleware)
   - SameSite cookie attribute: SameSite=Strict or Lax
   - Check Origin/Referer headers

5. SECURITY HEADERS (helmet.js)
   ────────────────────────────────
   const helmet = require('helmet');
   app.use(helmet());

   Sets:
     X-Content-Type-Options: nosniff
```

```
    X-Frame-Options: DENY
    Strict-Transport-Security: max-age=31536000
    Content-Security-Policy: ...
    X-XSS-Protection: 0 (deprecated, rely on CSP)

6. RATE LIMITING
   ____________
   const rateLimit = require('express-rate-limit');
   app.use('/api/', rateLimit({
     windowMs: 15 * 60 * 1000,   // 15 minutes
     max: 100,                     // 100 requests per window
     standardHeaders: true,
     legacyHeaders: false,
   }));

7. INPUT VALIDATION
   _________________
   Libraries: Joi, Zod, class-validator (NestJS)
   ALWAYS validate: type, length, range, format
   Validate at the BOUNDARY (API entry point)

   Zod example:
     const schema = z.object({
       email: z.string().email(),
       name: z.string().min(1).max(100),
       age: z.number().int().min(18).max(150),
     });
     const result = schema.safeParse(req.body);
     if (!result.success) return res.status(400).json(result.error);

8. DEPENDENCY VULNERABILITIES
   ___________________________
   npm audit                   → check for known vulnerabilities
   npm audit fix               → auto-fix
   npx npm-check-updates -u    → update dependencies
   Use Snyk, Dependabot, or Socket in CI/CD

9. SECRETS MANAGEMENT
   __________________
   - NEVER hardcode secrets in code
   - Use environment variables (process.env.DB_PASSWORD)
   - Use .env files for local dev ONLY (+ .gitignore)
   - Production: K8s Secrets, Vault, AWS Secrets Manager
   - dotenv for development: require('dotenv').config();
```

```
10. LOGGING & MONITORING
    ____________________
    - Never log secrets, tokens, passwords, PII
    - Log: request ID, user ID, action, result, duration
    - Structured logging (JSON): winston, pino
    - Monitor for anomalies (unusual error rates, auth failures)
```

---

14. Performance & Scaling

14.1 Cluster Module

```
// Fork one worker per CPU core — all share the same port

const cluster = require('cluster');
const os = require('os');

if (cluster.isPrimary) {
  const numCPUs = os.cpus().length;
  console.log(`Primary ${process.pid} forking ${numCPUs} workers`);

  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }

  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} died. Restarting...`);
    cluster.fork();
  });
} else {
  // Each worker runs the Express app
  const app = require('./app');
  app.listen(3000);
  console.log(`Worker ${process.pid} started`);
}

// In production: use PM2 instead of manual clustering
// pm2 start app.js -i max  ← starts one process per CPU
```

14.2 Worker Threads

```
// For CPU-intensive tasks — offload to a separate thread

// main.js
const { Worker } = require('worker_threads');

function runWorker(data) {
  return new Promise((resolve, reject) => {
    const worker = new Worker('./worker.js', { workerData: data });
    worker.on('message', resolve);
    worker.on('error', reject);
  });
}

app.get('/process', async (req, res) => {
  const result = await runWorker(req.body.data);
  res.json(result);
  // Event loop stays free while worker crunches numbers
});

// worker.js
const { workerData, parentPort } = require('worker_threads');
const result = heavyComputation(workerData);
parentPort.postMessage(result);
```

14.3 Caching

```
// ─── In-Memory Cache (single process) ───
const NodeCache = require('node-cache');
const cache = new NodeCache({ stdTTL: 300 }); // 5 min TTL

async function getUser(id) {
  const cached = cache.get(`user:${id}`);
  if (cached) return cached;

  const user = await db.findUser(id);
  cache.set(`user:${id}`, user);
  return user;
}

// ─── Redis (distributed cache — works across Pods) ───
const Redis = require('ioredis');
```

```
const redis = new Redis(process.env.REDIS_URL);

async function getUser(id) {
  const cached = await redis.get(`user:${id}`);
  if (cached) return JSON.parse(cached);

  const user = await db.findUser(id);
  await redis.set(`user:${id}`, JSON.stringify(user), 'EX', 300);
  return user;
}

// Cache invalidation patterns:
//   TTL-based: set expiry, data eventually refreshes
//   Write-through: update cache immediately on write
//   Cache-aside: app checks cache → miss → query DB → populate cache
//   Pub/Sub: notify all instances to invalidate
```

14.4 Performance Checklist

```
1. Use async/await for all I/O — never block the event loop
2. Stream large files (don't load into memory)
3. Connection pooling for DB
4. Cache frequently accessed data (Redis for multi-Pod)
5. Use compression middleware (compression package)
6. Set proper HTTP caching headers (ETag, Cache-Control)
7. Use reverse proxy (NGINX) for static files, SSL termination
8. Use PM2 or cluster module for multi-core utilization
9. Profile with clinic.js (doctor, flame, bubbleprof)
10. Monitor event loop lag — if > 100ms, something is blocking
11. Use worker_threads for CPU tasks
12. Increase UV_THREADPOOL_SIZE if doing many fs/crypto operations
13. Use HTTP/2 for multiplexed connections
14. Pagination — never return unbounded result sets
15. Select only needed fields (don't SELECT *)
```

---

15. Testing — Unit, Integration, E2E

15.1 Testing Stack

```
Test Runner:    Jest (most popular) or Vitest (faster, ESM-native)
Assertion:      Built into Jest
Mocking:        Jest mocks, sinon
HTTP Testing:   supertest (test Express routes without starting server)
E2E:            Playwright, Cypress (for frontend)
API E2E:        supertest or REST client libraries
Coverage:       Jest --coverage (Istanbul under the hood)
```

15.2 Unit Tests

```
// userService.test.js
const UserService = require('./userService');

describe('UserService', () => {
  let service;
  let mockUserRepo;

  beforeEach(() => {
    mockUserRepo = {
      findById: jest.fn(),
      create: jest.fn(),
    };
    service = new UserService(mockUserRepo);
  });

  describe('findById', () => {
    it('should return user when found', async () => {
      const mockUser = { id: 1, name: 'John' };
      mockUserRepo.findById.mockResolvedValue(mockUser);

      const result = await service.findById(1);

      expect(result).toEqual(mockUser);
      expect(mockUserRepo.findById).toHaveBeenCalledWith(1);
    });

    it('should throw NotFoundError when user missing', async () => {
      mockUserRepo.findById.mockResolvedValue(null);
```

```
      await expect(service.findById(999)).rejects.toThrow(NotFoundError);
    });
  });
});
```

15.3 Integration Tests (API)

```
const request = require('supertest');
const app = require('../app');

describe('GET /api/users', () => {
  it('should return users list', async () => {
    const res = await request(app)
      .get('/api/users')
      .set('Authorization', `Bearer ${testToken}`)
      .expect('Content-Type', /json/)
      .expect(200);

    expect(res.body.data).toBeInstanceOf(Array);
    expect(res.body.meta.total).toBeGreaterThan(0);
  });

  it('should return 401 without token', async () => {
    await request(app)
      .get('/api/users')
      .expect(401);
  });

  it('should paginate results', async () => {
    const res = await request(app)
      .get('/api/users?page=1&limit=5')
      .set('Authorization', `Bearer ${testToken}`)
      .expect(200);

    expect(res.body.data.length).toBeLessThanOrEqual(5);
  });
});
```

15.4 Testing Strategy

```
Test Pyramid:
  ┌─────────┐
  │   E2E   │  ← few, slow, expensive
  ├─────────┤
  │ Integration │  ← moderate, test routes + middleware + DB
  ├──────────────┤
  │    Unit Tests    │  ← many, fast, isolated (mock dependencies)
  └──────────────────┘

Coverage targets:
  - Business logic (services): 90%+
  - Controllers/routes: 80%+
  - Utils: 100%
  - Overall: 80%+ (but meaningful coverage > number chasing)

What to test:
  ✅ Business rules, edge cases, error paths
  ✅ Input validation
  ✅ Auth/authz middleware
  ✅ DB queries with actual test DB (integration)
  ❌ Don't test framework internals
  ❌ Don't test simple getters/setters
```

---

16. TypeScript with Node.js

```
TypeScript adds static typing to JavaScript.
Catches bugs at compile time, improves DX, better refactoring.

Setup:
  npm install typescript @types/node ts-node --save-dev
  npx tsc --init   ← generates tsconfig.json
```

16.1 tsconfig.json (Node.js optimized)

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "commonjs",
```

```
    "lib": ["ES2022"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

16.2 Essential TypeScript Patterns

```
// ─── Interfaces & Types ───
interface User {
  id: number;
  name: string;
  email: string;
  role: 'admin' | 'user' | 'moderator';  // union literal type
  createdAt: Date;
}

type CreateUserDto = Omit<User, 'id' | 'createdAt'>;      // exclude fields
type UpdateUserDto = Partial<CreateUserDto>;                // all fields
optional
type UserSummary = Pick<User, 'id' | 'name'>;              // only these fields

// ─── Generics ───
interface PaginatedResponse<T> {
  data: T[];
  meta: {
    total: number;
    page: number;
    limit: number;
  };
```

```
}

async function findAll(): Promise<PaginatedResponse<User>> { ... }

// ─── Enums (prefer const enums or union types) ───
const OrderStatus = {
  PENDING: 'PENDING',
  PROCESSING: 'PROCESSING',
  SHIPPED: 'SHIPPED',
  DELIVERED: 'DELIVERED',
} as const;

type OrderStatus = typeof OrderStatus[keyof typeof OrderStatus];
// 'PENDING' | 'PROCESSING' | 'SHIPPED' | 'DELIVERED'

// ─── Type Guards ───
function isAppError(error: unknown): error is AppError {
  return error instanceof AppError;
}

// ─── Express Typed (with @types/express) ───
import { Request, Response, NextFunction } from 'express';

interface AuthRequest extends Request {
  user: { id: number; role: string };
}

const getProfile = async (req: AuthRequest, res: Response) => {
  const userId = req.user.id;  // typed!
  // ...
};
```

---

17. Node.js Internals — Memory, GC, Diagnostics

17.1 Memory Model

```
V8 Heap Memory Layout:
  ┌──────────────────────────────────────┐
  │          New Space (Young Gen)       │  ← short-lived objects
  │  ┌──────────┬──────────────┐         │     Scavenge GC (fast, frequent)
  │  │ Semi-space│  Semi-space   │        │
```

```
  │  │  (from)  │   (to)        │     │     Default: 16MB
  │  └──────────┴───────────────┘     │
  ├──────────────────────────────────┤
  │          Old Space (Old Gen)      │  ← long-lived objects
  │                                   │     Mark-Sweep-Compact GC (slower)
  │                                   │     Default: ~1.4GB (64-bit)
  ├──────────────────────────────────┤
  │          Large Object Space       │  ← objects > 1MB
  ├──────────────────────────────────┤
  │          Code Space               │  ← JIT compiled code
  ├──────────────────────────────────┤
  │          Map Space                │  ← hidden classes (object shapes)
  └──────────────────────────────────┘

  Total heap default: ~1.5GB on 64-bit systems
  Override: node --max-old-space-size=4096 app.js

  In K8s: set max-old-space-size to ~75% of container memory limit
    Container: 512Mi → max-old-space-size=384
```

17.2 Garbage Collection

```
Scavenge (Young Gen):
  - Copies live objects from "from" space to "to" space
  - Dead objects are not copied → instantly freed
  - Fast (1-2ms), frequent
  - Objects surviving 2 scavenges are promoted to Old Gen

Mark-Sweep-Compact (Old Gen):
  - Mark: walk object graph, mark reachable objects
  - Sweep: free unreachable objects
  - Compact: defragment (move objects to eliminate gaps)
  - Slower (10-100ms), less frequent
  - Incremental: broken into small steps to avoid long pauses

Monitor GC:
  node --trace-gc app.js
  → Shows GC events, time, heap sizes

  Scavenge 2.3 (5.1) -> 1.8 (6.2) MB, 1.2 / 0.0 ms
  Mark-sweep 45.2 (67.3) -> 38.1 (67.3) MB, 23.4 / 0.0 ms
```

17.3 Memory Leaks

```
// COMMON LEAK PATTERNS:

// 1. Global variables (never garbage collected)
let cache = {};  // grows forever
function handleRequest(req) {
  cache[req.id] = processData(req);  // never cleaned up
}
// FIX: use LRU cache with max size, or WeakMap, or TTL

// 2. Event listeners not removed
function setup() {
  emitter.on('data', handler);    // called many times → listeners pile up
}
// FIX: emitter.once() or removeListener() when done

// 3. Closures holding references
function createHandler() {
  const hugeData = loadHugeDataset();   // 500MB
  return (req, res) => {
    // hugeData is captured in closure, never GC'd
    res.json(hugeData.find(item => item.id === req.params.id));
  };
}
// FIX: don't capture large data in closures, use DB queries instead

// 4. Forgotten timers
const interval = setInterval(() => {
  // runs forever if never cleared
}, 1000);
// FIX: clearInterval(interval) when no longer needed

// DETECT LEAKS:
//   node --inspect app.js  → Chrome DevTools → Memory tab → Heap snapshots
//   Take snapshot A → run load test → Take snapshot B → compare
//   Look for: objects that grew significantly between snapshots
//
//   process.memoryUsage()
//   → { rss, heapTotal, heapUsed, external, arrayBuffers }
//   If heapUsed keeps growing → leak
```

17.4 Diagnostics Tools

```
node --inspect app.js        → Chrome DevTools (chrome://inspect)
node --prof app.js           → V8 CPU profiling (generates .log)
node --trace-gc app.js       → GC events

clinic.js:
  npx clinic doctor -- node app.js
  npx clinic flame -- node app.js     ← flame graph
  npx clinic bubbleprof -- node app.js

Built-in:
  process.memoryUsage()     → heap stats
  process.cpuUsage()        → user/system CPU time
  process.uptime()          → seconds since start
  performance.now()         → high-res timer
  perf_hooks module         → detailed performance measurement
```

---

18. Microservices Patterns in Node.js

18.1 Communication

```
Synchronous:
  REST (HTTP/JSON):  axios, node-fetch, got
  gRPC:              @grpc/grpc-js (binary protocol, protobuf, fast)
  GraphQL:           Apollo Server, Mercurius

Asynchronous (event-driven):
  Message Queues:  RabbitMQ (amqplib), AWS SQS
  Event Streaming: Kafka (kafkajs), Redis Streams
  Pub/Sub:         Redis Pub/Sub, NATS

Choose:
  REST:    simple CRUD, external APIs, wide compatibility
  gRPC:    internal service-to-service, high throughput, streaming
  Events:  decoupled services, eventual consistency, saga pattern
```

18.2 Patterns

```
─── API Gateway ───
  Single entry point routes to backend microservices.
  Handles: auth, rate limiting, routing, aggregation.
  Node.js: Express gateway, custom Express proxy, Kong, AWS API Gateway.

─── Circuit Breaker ───
  Prevents cascading failures when downstream service is down.
  States: Closed (normal) → Open (fail fast) → Half-Open (test)
  Libraries: opossum

  const CircuitBreaker = require('opossum');
  const breaker = new CircuitBreaker(callExternalService, {
    timeout: 3000,          // fail if no response in 3s
    errorThresholdPercentage: 50,  // open if 50% of requests fail
    resetTimeout: 10000,    // try again after 10s
  });
  const result = await breaker.fire(params);

─── Rate Limiting ───
  Token bucket, sliding window, fixed window.
  In-memory: express-rate-limit (single instance)
  Distributed: rate-limit-redis (across Pods)

─── Retry with Backoff ───
  Libraries: async-retry, p-retry
  const retry = require('async-retry');
  const result = await retry(async () => {
    return await callUnreliableService();
  }, { retries: 3, minTimeout: 1000, factor: 2 });
  // Retries: 1s, 2s, 4s

─── Event Sourcing / CQRS ───
  Store events instead of current state.
  Separate read models (optimized for queries) from write models.
  Node.js: eventstore, custom with Kafka.

─── Health Check ───
  GET /health → { status: 'ok', uptime: 12345, checks: { db: 'ok', redis: 'ok' }
}
  K8s liveness/readiness probes hit this endpoint.
```

19. Node.js + Docker + K8s (Production)

19.1 Dockerfile

```
# ─── Multi-stage build ───
FROM node:20-alpine AS builder
WORKDIR /app

# Install dependencies first (layer caching)
COPY package*.json ./
RUN npm ci --only=production

# Copy source and build (if TypeScript)
COPY . .
RUN npm run build

# ─── Production image ───
FROM node:20-alpine
WORKDIR /app

# Non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Copy only what's needed
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./

USER appuser
EXPOSE 3000

# Use node directly, not npm start (signal handling)
CMD ["node", "dist/server.js"]
```

```
Key points:
  - Alpine base: 50MB vs 900MB for full Ubuntu image
  - npm ci (not npm install): deterministic, uses lock file
  - Multi-stage: build tools stay in builder, not in production image
  - Non-root: matches K8s runAsNonRoot
  - CMD ["node", ...] not CMD ["npm", "start"]:
      npm spawns a shell → PID 1 is npm, not node
```

```
      SIGTERM sent to PID 1 → npm doesn't forward to node → ungraceful shutdown
  - .dockerignore: node_modules, .git, .env, tests, docs
```

19.2 Graceful Shutdown

```
// CRITICAL for K8s. When Pod is terminated:
//   1. K8s sends SIGTERM to PID 1
//   2. App has terminationGracePeriodSeconds (default 30s) to shut down
//   3. If still running → K8s sends SIGKILL (force kill)

const server = app.listen(3000);

async function gracefulShutdown(signal) {
  console.log(`Received ${signal}. Shutting down gracefully...`);

  // 1. Stop accepting new connections
  server.close(async () => {
    console.log('HTTP server closed.');

    // 2. Close DB connections
    await prisma.$disconnect();

    // 3. Close Redis
    await redis.quit();

    // 4. Close message queue connections
    await rabbitChannel.close();
    await rabbitConnection.close();

    console.log('All connections closed. Exiting.');
    process.exit(0);
  });

  // Force exit if graceful shutdown takes too long
  setTimeout(() => {
    console.error('Forced shutdown after timeout');
    process.exit(1);
  }, 25000);  // less than K8s terminationGracePeriodSeconds
}

process.on('SIGTERM', () => gracefulShutdown('SIGTERM'));
process.on('SIGINT', () => gracefulShutdown('SIGINT'));
```

19.3 K8s Deployment Manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-node-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-node-api
  template:
    metadata:
      labels:
        app: my-node-api
    spec:
      containers:
        - name: api
          image: myregistry/my-node-api:v1.2.3
          ports:
            - containerPort: 3000
          env:
            - name: NODE_ENV
              value: "production"
            - name: DB_HOST
              valueFrom:
                configMapKeyRef:
                  name: api-config
                  key: DB_HOST
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: api-secrets
                  key: DB_PASSWORD
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              memory: "512Mi"
              # No CPU limit — avoid throttling
          livenessProbe:
            httpGet:
              path: /health
              port: 3000
```

```
            initialDelaySeconds: 5
            periodSeconds: 15
          readinessProbe:
            httpGet:
              path: /health/ready
              port: 3000
            initialDelaySeconds: 3
            periodSeconds: 10
          lifecycle:
            preStop:
              exec:
                command: ["sleep", "5"]
                # Wait for kube-proxy to update iptables
          securityContext:
            runAsNonRoot: true
            readOnlyRootFilesystem: true
            allowPrivilegeEscalation: false
      terminationGracePeriodSeconds: 30
```

19.4 Node.js in K8s — Memory Sizing

```
Container limit: 512Mi

  node --max-old-space-size=384 dist/server.js

  Breakdown:
    V8 heap (old space):  384Mi (max-old-space-size)
    V8 heap (young gen):  ~16Mi
    Native/C++ addons:    ~50Mi
    Buffers/streams:      ~30Mi
    Overhead:             ~32Mi
    Total:                ~512Mi ✅

  If heapUsed approaches max-old-space-size → V8 throws:
    FATAL ERROR: CALL_AND_RETRY_LAST Allocation failed - JavaScript heap out of
memory

  If total RSS exceeds container limit → K8s OOMKills the Pod (exit code 137)

  Set max-old-space-size to ~75% of container memory limit.
```

---

20. Common Interview Questions & Answers

Q1: What is the event loop and how does it work?

```
Node.js uses a single-threaded event loop (libuv) to handle async I/O.
It cycles through phases: timers → pending → poll → check → close.
Between phases, it drains nextTick and microtask (Promise) queues.
I/O operations are delegated to the OS (epoll/kqueue) or libuv thread pool.
This allows thousands of concurrent connections without thread overhead.
```

Q2: How does Node.js handle concurrency if it's single-threaded?

```
The main thread handles JS execution (event loop).
I/O is non-blocking — OS handles network, libuv thread pool handles fs/crypto.
When I/O completes, callbacks queue up and run on the event loop.
One thread handles scheduling, OS/threads handle the actual work.
For CPU tasks: worker_threads create real OS threads.
For multi-core: cluster module forks multiple processes.
```

Q3: What's the difference between process.nextTick() and setImmediate()?

```
process.nextTick(): fires BEFORE the next event loop phase (between phases).
setImmediate(): fires in the CHECK phase (after poll).
nextTick has higher priority — runs first.
Danger: recursive nextTick() starves the event loop (I/O never gets processed).
Best practice: prefer setImmediate() for deferring work.
```

Q4: How do you prevent memory leaks?

```
1. Avoid global caches without size limits (use LRU cache)
2. Remove event listeners when done (removeListener, once())
3. Don't capture large objects in closures
4. Clear intervals/timeouts when not needed
5. Use WeakMap/WeakSet for object references that should be GC'd
6. Monitor: process.memoryUsage(), heap snapshots via --inspect
7. In production: set --max-old-space-size, alert on growing RSS
```

Q5: Explain streams and when to use them.

```
Streams process data in chunks instead of loading all into memory.
Four types: Readable, Writable, Duplex, Transform.
Use for: file operations, HTTP responses, data pipelines, CSV parsing.
Key benefit: constant memory with any file size.
pipe() handles backpressure automatically.
Modern: pipeline() from stream/promises with proper error handling.
```

Q6: How do you handle errors in async code?

```
Async/await: wrap in try/catch
Promises: attach .catch()
Express: asyncHandler wrapper that catches and forwards to error middleware
Global: process.on('unhandledRejection') — log and decide on exit
Custom error classes with statusCode for clean API responses.
Never swallow errors silently.
```

Q7: What's the difference between CommonJS and ES Modules?

```
CJS: require/module.exports, synchronous, runtime-resolved, default in Node
ESM: import/export, async, parse-time static analysis, tree-shakeable
ESM supports top-level await, CJS doesn't.
ESM uses import.meta.url instead of __dirname.
CJS caches by filename (singleton behavior).
Use ESM for new projects (better tooling, tree-shaking, standardized).
```

Q8: How do you scale a Node.js application?

```
Vertical: cluster module or PM2 to use all CPU cores (one process per core).
Horizontal: multiple Pods in K8s behind a Service (load balanced).
Stateless design: no in-process state → any Pod can handle any request.
Session/cache: externalize to Redis.
CPU tasks: worker_threads or separate microservice.
DB: connection pooling, read replicas, caching.
Async: message queues (RabbitMQ, Kafka) for decoupling.
```

Q9: Explain middleware in Express.

```
Middleware: function(req, res, next) that runs in order of registration.
Each can: modify req/res, end the request, or call next().
Types: application-level (app.use), route-level, error-handling (4 args), built-
in.
Execution order matters — registered top to bottom.
Error middleware has 4 params: (err, req, res, next).
Common: auth, logging, body parsing, CORS, rate limiting, validation.
```

Q10: How would you design a REST API for a large application?

```
Feature-based folder structure (not layer-based).
Controller → Service → Repository separation.
Input validation at boundary (Joi/Zod).
Consistent error responses with status codes.
Pagination, filtering, sorting on list endpoints.
JWT auth + RBAC middleware.
Rate limiting per client.
API versioning (/api/v1/).
OpenAPI/Swagger documentation.
Integration tests with supertest.
```

Q11: What is libuv's thread pool used for?

```
Operations that can't be done async at the OS level:
  - File system operations (fs.readFile, fs.stat)
  - DNS lookups (dns.lookup, NOT dns.resolve)
  - Crypto operations (pbkdf2, scrypt, randomBytes)
  - Zlib compression
Default: 4 threads. Tunable: UV_THREADPOOL_SIZE=16 (max 1024).
Network I/O does NOT use the thread pool — uses OS async (epoll/kqueue).
```

Q12: How do you handle graceful shutdown in K8s?

```
1. Listen for SIGTERM signal
2. Stop accepting new connections (server.close())
3. Finish in-flight requests
4. Close DB connections, Redis, message queues
5. Exit process
```

```
Use preStop hook with sleep 5 (allow kube-proxy iptables update).
Set timeout < terminationGracePeriodSeconds (default 30s).
Use node directly in CMD, not npm (npm doesn't forward signals).
```

Q13: Cluster module vs Worker Threads?

```
Cluster module:
  - Forks separate PROCESSES (each has own V8 instance, own memory)
  - Shares the same port via IPC
  - Use for: scaling HTTP servers across CPU cores
  - Communication: IPC (process.send/on)

Worker Threads:
  - Creates threads within the SAME process
  - Can share memory (SharedArrayBuffer)
  - Use for: CPU-intensive tasks (parsing, compression, ML)
  - Communication: parentPort.postMessage/on

Don't confuse: cluster = multi-process, workers = multi-thread.
```

Q14: What's the N+1 query problem and how do you solve it?

```
Problem: fetching a list of users, then querying orders for EACH user.
  1 query for users + N queries for orders = N+1 queries.

Solutions:
  1. Eager loading: ORM join/include
     prisma.user.findMany({ include: { orders: true } })
  2. Batch loading: DataLoader (collects IDs, makes one query)
     const loader = new DataLoader(ids => batchGetOrders(ids));
  3. Raw SQL: JOIN query
     SELECT u.*, o.* FROM users u LEFT JOIN orders o ON u.id = o.user_id
```

Q15: How do you secure a Node.js API?

```
1. Input validation (Joi/Zod) — never trust client input
2. Parameterized queries — prevent SQL injection
3. bcrypt for password hashing (cost factor 12+)
4. JWT with short-lived access tokens, httpOnly cookies
5. helmet.js for security headers
6. Rate limiting (express-rate-limit)
```

```
7. CORS configuration (whitelist origins)
8. npm audit for dependency vulnerabilities
9. HTTPS everywhere (TLS termination at Ingress/LB)
10. Principle of least privilege (RBAC, minimal permissions)
```

Quick Reference: Node.js vs Java (for Full Stack Context)

```
┌──────────────────────┬──────────────────────────┬──────────────────────────┐
│ Aspect               │ Java (Spring Boot)       │ Node.js (Express/Nest)   │
├──────────────────────┼──────────────────────────┼──────────────────────────┤
│ Concurrency model    │ Thread-per-request       │ Single thread + event loop │
│ Blocking I/O         │ Default (thread blocks)  │ Non-blocking (async)     │
│ Performance profile  │ CPU-heavy workloads      │ I/O-heavy workloads      │
│ Startup time         │ 10-60s (Spring)          │ 1-3s                     │
│ Memory               │ 256MB-1GB+ (JVM overhead)│ 50-200MB (lighter)       │
│ Type safety          │ Built-in (Java is typed) │ Opt-in (TypeScript)      │
│ Package manager      │ Maven/Gradle             │ npm/yarn/pnpm            │
│ ORM                  │ JPA/Hibernate            │ Prisma/TypeORM/Sequelize│
│ DI framework         │ Spring IoC               │ NestJS (inversify, tsyringe)│
│ Testing              │ JUnit + Mockito          │ Jest + supertest         │
│ Build artifact       │ .jar                     │ Docker image (node + source) │
│ K8s probes           │ Actuator                 │ /health endpoint         │
│ Graceful shutdown    │ server.shutdown=graceful  │ SIGTERM handler +
server.close() │
│ When to choose       │ Enterprise, complex domain│ Real-time, BFF, APIs    │
│                      │ heavy computation         │ microservices, streaming │
└──────────────────────┴──────────────────────────┴──────────────────────────┘
```

---

21. The Event Loop — Deep Dive (Story Mode + Technical Mode)

Part 1: Story Mode — "The Restaurant That Never Sleeps"

```
Imagine a restaurant called "Node Café" with ONE waiter named EVAN (the Event
Loop).
```

```
THE SETUP:
  - Evan is the ONLY waiter. There is no second waiter.
  - But there's a KITCHEN with 4 cooks (the libuv thread pool).
  - And a DELIVERY SERVICE outside (the OS async I/O — epoll/kqueue).
  - There's a PRIORITY BUZZER system for urgent tasks.

THE RESTAURANT OPENS. Here's what happens over one busy evening.

═══════════════════════════════════════════════════════════════════
SCENE 1: A customer walks in (an HTTP request arrives)
═══════════════════════════════════════════════════════════════════

  Customer: "I'd like the user profile page."

  Evan DOESN'T go to the kitchen himself. He writes the order on a slip:
    "Query the database for user #42"

  He hands the slip to the DELIVERY SERVICE (OS async I/O for network calls).
  The delivery service will call the database over the network.

  Evan is FREE. He doesn't wait. He turns to the next customer.

  This is NON-BLOCKING I/O.
  The waiter (event loop) never stands around waiting for food (I/O results).

═══════════════════════════════════════════════════════════════════
SCENE 2: Another customer walks in immediately
═══════════════════════════════════════════════════════════════════

  Customer 2: "I need to read a large CSV file from disk."

  Evan writes the order: "Read /data/report.csv"
  He hands it to one of the 4 KITCHEN COOKS (libuv thread pool).

  Why the kitchen instead of delivery? Because FILE I/O can't be done
  async at the OS level on all platforms. It needs a dedicated worker.

  Evan is FREE again. He turns to the next customer.

  Kitchen cooks handle: file reads, file writes, DNS lookups, crypto, zlib.
  Delivery service handles: network sockets, TCP, HTTP, database connections.

═══════════════════════════════════════════════════════════════════
SCENE 3: Customer 3 says "Remind me in 5 seconds" (setTimeout)
═══════════════════════════════════════════════════════════════════
```

```
  Customer 3: "Set a timer. After 5 seconds, bring me coffee."

  Evan writes on the TIMER BOARD on the wall:
    "5000ms from now → bring coffee to table 3"

  Evan doesn't stare at the clock. He moves on.

  The timer board is checked at the START of each loop cycle (TIMERS phase).
  When time is up, Evan sees the note on his next pass and delivers coffee.

  KEY INSIGHT: If Evan is busy serving a massive plate (a CPU-heavy sync task),
  he can't check the timer board. The coffee arrives LATE.
  setTimeout(fn, 5000) means "no EARLIER than 5000ms", not "exactly at 5000ms."

═══════════════════════════════════════════════════════════════════
SCENE 4: The delivery service rings the bell (DB query returns)
═══════════════════════════════════════════════════════════════════

  *DING* — The delivery service says:
  "User #42 data is ready. Here's the result."

  Evan was just finishing with customer 3.
  He picks up the result and delivers it to customer 1.
  Customer 1 gets their response. Request complete.

  This is the POLL PHASE — Evan checks the delivery window for completed I/O.

═══════════════════════════════════════════════════════════════════
SCENE 5: VIP customer enters (process.nextTick)
═══════════════════════════════════════════════════════════════════

  A VIP buzzes the PRIORITY BUZZER.

  Rule: The priority buzzer gets checked BETWEEN EVERY task.
  Even if Evan just finished one delivery and is about to check timers,
  he checks the buzzer FIRST.

  process.nextTick = the HIGHEST priority buzzer.
  Promise callbacks = the second priority buzzer (microtask queue).

  The VIP always cuts in line. Always.

  DANGER: If the VIP keeps buzzing endlessly (recursive nextTick),
  Evan NEVER gets to check timers, I/O, or anything else.
```

```
  The restaurant is stuck serving one demanding VIP forever.
  This is called STARVATION.

═══════════════════════════════════════════════════════════════
SCENE 6: "Check on me after each round" (setImmediate)
═══════════════════════════════════════════════════════════════

  Customer 6: "After you finish your current round of duties, come see me."

  setImmediate = "Run this callback in the CHECK phase —
  right after you've handled all I/O in this cycle."

  It's like saying: "I'm not urgent, but handle me in THIS loop iteration,
  after you've dealt with the incoming orders."

═══════════════════════════════════════════════════════════════
SCENE 7: "Repeat every 2 seconds" (setInterval)
═══════════════════════════════════════════════════════════════

  Customer 7: "Every 2 seconds, bring me a glass of water."

  Evan writes on the timer board:
    "Every 2000ms → water to table 7 (repeating)"

  Every time the timer fires, Evan delivers water.
  But if Evan is busy (blocked), the interval doesn't "stack up."
  It just fires when Evan next checks the timer board.

  setInterval(fn, 2000) ≠ "exactly every 2s"
  It means "schedule the callback at least 2s apart, when the event loop gets to
it."

═══════════════════════════════════════════════════════════════
SCENE 8: Evan's complete routine (ONE full loop cycle)
═══════════════════════════════════════════════════════════════

  Evan's shift follows a STRICT routine. Every cycle:

  ① Check PRIORITY BUZZER (nextTick + Promises)
  ② Check TIMER BOARD (setTimeout/setInterval due?)
     → Deliver any that are due
     → Check priority buzzer again

  ③ Check PENDING SLIPS (deferred I/O callbacks from last round)
     → Check priority buzzer again
```

```
  ④ Stand at the DELIVERY WINDOW (poll phase)
     → Pick up all completed I/O results
     → Deliver each one to the correct customer
     → If nothing is ready, WAIT here (this is where Evan rests)
     → But if timers are due or setImmediate is pending → move on
     → Check priority buzzer again

  ⑤ Handle "AFTER-ROUND" requests (setImmediate / check phase)
     → Check priority buzzer again

  ⑥ Handle CLOSING TABS (close callbacks: socket.on('close'))
     → Check priority buzzer again

  ⑦ Go back to ① and repeat.

  If NO customers remain and NO timers are set → Evan closes the restaurant.
  (process exits)

═══════════════════════════════════════════════════════════════
SCENE 9: THE DISASTER — A customer asks Evan to cook (CPU blocking)
═══════════════════════════════════════════════════════════════

  Customer 9: "Calculate fibonacci(45) for me."

  Evan can't hand this to the kitchen — it's pure computation,
  not I/O. It has to happen in JavaScript. Evan starts computing...

  For 5 seconds, Evan is FROZEN. He can't:
    - Check timers (coffees go undelivered)
    - Pick up I/O results (database responses pile up)
    - Serve any other customer (everyone waits)

  The ENTIRE restaurant stalls because the ONE waiter is doing math.

  SOLUTION: Hire a FREELANCER (worker_thread).
    Evan writes: "Hey freelancer, compute fibonacci(45)."
    Freelancer does the work in a SEPARATE room (separate thread).
    When done, freelancer rings a bell, Evan delivers the result.
    Evan stays free the entire time.

═══════════════════════════════════════════════════════════════
THE MORAL:
  - Evan (event loop) is ONE person — never block him
  - I/O goes to delivery service (OS) or kitchen (thread pool) — non-blocking
```

```
  - Timers are checked periodically — not precise
  - VIP buzzer (nextTick/Promises) always cuts in line
  - CPU work blocks EVERYTHING — offload it
  - The restaurant works because Evan DELEGATES I/O, he never DOES I/O
═══════════════════════════════════════════════════════════════════════
```

Part 2: Technical Deep Dive — Every Scenario Traced

2.1 The Event Loop Phases — Complete Diagram

```
┌──────────────────────────────────────────────────────────────────┐
│                    EVENT LOOP TICK                               │
│                                                                  │
│  ┌──────────┐    Between EVERY phase transition:               │
│  │          │    ┌──────────────────────────────┐               │
│  │  TIMERS  │    │ 1. Drain process.nextTick queue │            │
│  │          │    │ 2. Drain microtask queue        │            │
│  └──────────┘    │    (Promise .then/.catch/.finally)│          │
│       │          └──────────────────────────────┘               │
│  ┌────▼─────┐                                                   │
│  │ PENDING  │                                                   │
│  │CALLBACKS │                                                   │
│  └──────────┘                                                   │
│       │                                                         │
│  ┌────▼─────┐                                                   │
│  │  IDLE /  │    (internal — skip)                              │
│  │ PREPARE  │                                                   │
│  └──────────┘                                                   │
│       │                                                         │
│  ┌────▼─────┐                                                   │
│  │   POLL   │    ← THIS IS WHERE NODE SPENDS MOST TIME          │
│  │          │      Waiting for and processing I/O events        │
│  └──────────┘                                                   │
│       │                                                         │
│  ┌────▼─────┐                                                   │
│  │  CHECK   │    ← setImmediate callbacks                       │
│  └──────────┘                                                   │
│       │                                                         │
│  ┌────▼─────┐                                                   │
│  │  CLOSE   │    ← socket.on('close') etc.                      │
│  └──────────┘                                                   │
│       │                                                         │
```

```
  │       └──────────── back to TIMERS ──────────────────────────│
  └──────────────────────────────────────────────────────────────┘
```

2.2 Scenario: HTTP Request with DB Query

```
const http = require('http');
const { Pool } = require('pg');
const pool = new Pool();

const server = http.createServer(async (req, res) => {   // ← STEP 1
  console.log('Request received');                        // ← STEP 2

  const result = await pool.query('SELECT * FROM users'); // ← STEP 3

  console.log('Query done');                              // ← STEP 5
  res.end(JSON.stringify(result.rows));                   // ← STEP 6
});

server.listen(3000);
```

```
TRACE (what happens under the hood):

STEP 1: HTTP request arrives
  └─ OS notifies libuv (via epoll/kqueue) that data is ready on the TCP socket
  └─ libuv places callback in the POLL QUEUE
  └─ Event loop is in POLL phase → executes the request handler

STEP 2: console.log('Request received')
  └─ Synchronous. Runs immediately on the call stack. Prints to stdout.

STEP 3: pool.query('SELECT * FROM users')
  └─ pg driver sends SQL over TCP socket to PostgreSQL (ASYNC network I/O)
  └─ This is handled by OS async I/O (epoll/kqueue) — NOT the thread pool
  └─ Returns a Promise. await suspends this function.
  └─ Event loop is FREE — goes back to poll phase, can handle other requests.

  ... time passes while PostgreSQL processes the query ...

STEP 4: DB response arrives (invisible in code)
  └─ OS signals libuv: "data ready on PostgreSQL TCP socket"
  └─ libuv places the I/O callback in the POLL QUEUE
  └─ When event loop reaches poll phase → executes the pg callback
  └─ pg resolves the Promise
  └─ Promise resolution goes to MICROTASK QUEUE
```

```
  └─ After current poll callbacks → drain microtask queue
  └─ The awaited function RESUMES at step 5

STEP 5: console.log('Query done')
  └─ Synchronous. Prints immediately.

STEP 6: res.end(JSON.stringify(result.rows))
  └─ Node writes response to TCP socket (async — buffered by OS)
  └─ HTTP response sent to client.

TOTAL BLOCKING TIME ON EVENT LOOP: ~microseconds
  The DB query took 50ms, but the event loop was FREE for all 50ms.
  It could serve OTHER requests during that time.
```

2.3 Scenario: File Read (Uses Thread Pool)

```
const fs = require('fs');

console.log('A');                                    // 1

fs.readFile('/data/big-file.json', 'utf8', (err, data) => {
  console.log('B - file read complete');             // 5
});

console.log('C');                                    // 2

setTimeout(() => {
  console.log('D - timer');                          // 3 or 4
}, 0);

setImmediate(() => {
  console.log('E - immediate');                      // 3 or 4
});

console.log('F');                                    // 2 (after C)
```

```
TRACE:

EXECUTION ORDER:
  A                    ← sync, immediate
  C                    ← sync, immediate
  F                    ← sync, immediate
  --- sync code done, event loop starts ---
  D or E               ← timer vs immediate (order undefined in main module)
```

```
  E or D                ← whichever didn't go first
  B - file read complete ← arrives when thread pool worker finishes

WHY:
  1. All sync code runs first (A, C, F) — this is the initial execution
  2. Event loop starts:
     - setTimeout(fn, 0): scheduled for TIMERS phase
     - setImmediate(fn): scheduled for CHECK phase
     - In the main module (not inside I/O callback):
       Timer vs Immediate order depends on process performance.
       If loop starts before 1ms has passed → timers has nothing → check runs
first (E)
       If loop starts after 1ms → timer fires first (D)
       THIS IS UNDEFINED BEHAVIOR in main module.
  3. fs.readFile: sent to libuv THREAD POOL
     - One of the 4 worker threads reads the file
     - When done, callback is queued for POLL phase
     - Arrives later (could be many loop cycles later for big files)

INSIDE an I/O callback, the order IS deterministic:
  fs.readFile(__filename, () => {
    setTimeout(() => console.log('timeout'), 0);
    setImmediate(() => console.log('immediate'));
  });
  // ALWAYS: immediate, timeout
  // Because: we're in poll phase → check phase is NEXT → timers come AFTER
```

2.4 Scenario: The Full Mix — setTimeout, setInterval, Promise, nextTick, I/O

```
const fs = require('fs');

// ═══════════════════════════════════════════
// SCRIPT START — all sync code runs first
// ═══════════════════════════════════════════

console.log('1 - script start');                        // SYNC

setTimeout(() => {
  console.log('2 - setTimeout 0ms');                    // TIMERS

  Promise.resolve().then(() => {
    console.log('3 - promise inside setTimeout');       // MICROTASK (after timer
cb)
  });
```

```
  process.nextTick(() => {
    console.log('4 - nextTick inside setTimeout');      // NEXTTICK (before
promise)
  });
}, 0);

setImmediate(() => {
  console.log('5 - setImmediate');                      // CHECK
});

fs.readFile(__filename, () => {
  console.log('6 - I/O complete');                      // POLL

  setTimeout(() => {
    console.log('7 - setTimeout inside I/O');           // TIMERS (next cycle)
  }, 0);

  setImmediate(() => {
    console.log('8 - setImmediate inside I/O');         // CHECK (this cycle)
  });

  process.nextTick(() => {
    console.log('9 - nextTick inside I/O');             // NEXTTICK (immediate)
  });
});

Promise.resolve().then(() => {
  console.log('10 - promise');                          // MICROTASK
});

process.nextTick(() => {
  console.log('11 - nextTick');                         // NEXTTICK

  Promise.resolve().then(() => {
    console.log('12 - promise inside nextTick');        // MICROTASK
  });
});

console.log('13 - script end');                         // SYNC
```

```
OUTPUT (deterministic for the sync + microtask part):

  1  - script start           ← sync
  13 - script end             ← sync
```

```
  ─── sync code done ───
  11 - nextTick                ← nextTick queue (drained first)
  12 - promise inside nextTick ← microtask from inside nextTick
  10 - promise                 ← microtask queue
  ─── event loop starts ───
  2  - setTimeout 0ms          ← timers phase (or after immediate, see note)
  4  - nextTick inside setTimeout  ← nextTick between phases
  3  - promise inside setTimeout   ← microtask between phases
  5  - setImmediate            ← check phase
  ─── I/O completes (when fs finishes) ───
  6  - I/O complete            ← poll phase
  9  - nextTick inside I/O     ← nextTick (immediate, between phases)
  8  - setImmediate inside I/O ← check phase (this cycle)
  7  - setTimeout inside I/O   ← timers phase (next cycle)

NOTES:
  - 2 and 5 may swap in main module (see 2.3 explanation)
  - 11 always before 10: nextTick has higher priority than Promises
  - 12 fires right after 11: nextTick callback created a Promise,
    which goes to microtask queue, drained before moving on
  - 9 fires right after 6: nextTick inside I/O callback is drained
    before the event loop moves to the next phase
  - 8 before 7: inside I/O callback, setImmediate (check) always
    fires before setTimeout (timers), because poll → check → timers
```

2.5 Scenario: setInterval — Drift and Overlap

```
let count = 0;

const interval = setInterval(() => {
  count++;
  console.log(`Tick ${count} at ${Date.now()}`);

  if (count === 1) {
    // Simulate blocking work on first tick
    const start = Date.now();
    while (Date.now() - start < 150) {} // block 150ms
  }

  if (count >= 5) clearInterval(interval);
}, 100);
```

```
EXPECTED (if perfectly precise):
  Tick 1 at T+100
```

```
  Tick 2 at T+200
  Tick 3 at T+300
  Tick 4 at T+400
  Tick 5 at T+500

ACTUAL:
  Tick 1 at T+100    ← fires on time
  (blocks for 150ms — event loop frozen)
  Tick 2 at T+250    ← should be T+200, but loop was blocked until T+250
  Tick 3 at T+350    ← 100ms after tick 2 (interval doesn't "catch up")
  Tick 4 at T+450
  Tick 5 at T+550

KEY FACTS:
  - setInterval does NOT guarantee precise intervals
  - If the callback takes longer than the interval, the next invocation
    is queued but does NOT stack up or run concurrently
  - The gap between fires is AT LEAST the interval, possibly more
  - If you need precise timing: use setTimeout recursively
    (schedule next tick at end of current tick)

RECURSIVE setTimeout (more precise):
  function tick() {
    doWork();
    setTimeout(tick, 100);  // 100ms gap AFTER work completes
  }
  setTimeout(tick, 100);
  // This guarantees 100ms of REST between each execution
  // setInterval guarantees 100ms from START to START (which can overlap)
```

2.6 Scenario: HTTP Server Under Load — Multiple Concurrent Requests

```
const http = require('http');
const { Pool } = require('pg');
const pool = new Pool({ max: 20 });

const server = http.createServer(async (req, res) => {
  const start = Date.now();

  // Step 1: DB query (50ms network I/O)
  const users = await pool.query('SELECT * FROM users LIMIT 100');

  // Step 2: External API call (200ms network I/O)
  const enriched = await fetch('http://enrichment-service/api/enrich', {
```

```
    method: 'POST',
    body: JSON.stringify(users.rows),
  });
  const data = await enriched.json();

  // Step 3: Write response
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(data));

  console.log(`Request served in ${Date.now() - start}ms`);
});
```

```
100 REQUESTS ARRIVE SIMULTANEOUSLY. What happens?

REQUEST TIMELINE:

  T=0ms: 100 requests arrive
    └─ OS accepts all 100 TCP connections (OS handles this, not Node)
    └─ libuv places 100 callbacks in the poll queue
    └─ Event loop processes them ONE BY ONE (single thread!)
    └─ But each one is FAST (just sets up an async DB query)

  T=0-1ms: Event loop starts processing all 100 request callbacks
    └─ For each: sends SQL query over TCP to PostgreSQL
    └─ Each pool.query() returns a Promise → await suspends the handler
    └─ Event loop moves to the next request handler
    └─ All 100 DB queries are IN FLIGHT simultaneously
    └─ They use 20 connections from the pool (remaining 80 queue inside pg)

  T=0-1ms: Event loop is FREE — all sync work done, all I/O delegated
    └─ Enters poll phase, WAITS for I/O events

  T=50ms: DB responses start arriving
    └─ OS signals "data ready" on pg TCP sockets
    └─ Event loop wakes up, processes each response:
        → Promise resolves → microtask → async function resumes
        → Sends fetch() to enrichment service → new I/O → suspends again
    └─ Event loop is FREE again

  T=250ms: Enrichment API responses arrive
    └─ Same pattern: Promise resolves, function resumes, res.end() called
    └─ Response written to TCP socket (OS handles sending)

  RESULT:
    Single thread handled 100 requests in ~250ms total
```

```
    Each request took ~250ms (DB wait + API wait)
    Event loop was IDLE for ~249ms out of 250ms
    Total CPU time: ~1ms (just scheduling callbacks)

THIS IS WHY NODE.JS HANDLES HIGH CONCURRENCY:
  The event loop doesn't DO the waiting — the OS does.
  Node just schedules and dispatches.
  10,000 concurrent requests ≈ 10,000 async I/O operations in flight.
  Event loop overhead per request: microseconds.
```

2.7 Scenario: Promise Microtask Queue Starvation

```
// ❌ DANGER: This starves the event loop

function recursivePromise() {
  Promise.resolve().then(() => {
    console.log('microtask');
    recursivePromise();  // creates another microtask → infinite loop
  });
}

recursivePromise();

setTimeout(() => {
  console.log('This NEVER prints');  // timer callback never reached
}, 100);
```

```
WHY THIS IS DANGEROUS:

  The microtask queue is drained COMPLETELY between every phase.
  If a microtask creates another microtask, it's added to the SAME queue.
  The queue never empties → event loop never advances.

  Result:
    - "microtask" prints forever
    - setTimeout callback NEVER fires
    - I/O NEVER gets processed
    - Server is DEAD

  Same problem with process.nextTick:
    function recursiveNextTick() {
      process.nextTick(() => {
        recursiveNextTick();  // nextTick queue never empties
      });
```

```
    }

  FIX: Use setImmediate for recursive scheduling:
    function safeRecursion() {
      setImmediate(() => {
        doWork();
        safeRecursion();  // schedules for CHECK phase — I/O still gets processed
      });
    }

  PRIORITY RANKING (highest to lowest):
    1. Synchronous code (call stack)
    2. process.nextTick queue
    3. Microtask queue (Promise.then)
    4. Macrotask: timers (setTimeout, setInterval)
    5. Macrotask: I/O callbacks (poll phase)
    6. Macrotask: setImmediate (check phase)
    7. Close callbacks
```

2.8 Scenario: Real Production — Express Request with Everything

```
app.get('/api/orders/:id', authenticate, async (req, res) => {
  // ═══════ What happens under the hood ═══════

  // 1. authenticate middleware ran FIRST (sync or async)
  //    → JWT verify is SYNC (crypto — but jwt.verify is CPU-bound,
  //      runs on event loop. For RS256, consider async verify)

  // 2. Route handler starts
  const orderId = req.params.id;

  // 3. DB query — ASYNC (network I/O via OS)
  //    Event loop: FREE while waiting
  const order = await db.query(
    'SELECT * FROM orders WHERE id = $1', [orderId]
  );
  //    Promise resolves → microtask → resumes here

  if (!order) {
    // 4. res.status().json() — SYNC (writes to buffer, OS sends async)
    return res.status(404).json({ error: 'Not found' });
  }

  // 5. Redis cache check — ASYNC (network I/O via OS)
```

```
  //    Event loop: FREE while waiting
  const cached = await redis.get(`shipping:${orderId}`);

  let shippingStatus;
  if (cached) {
    shippingStatus = JSON.parse(cached);  // SYNC — CPU (fast, microseconds)
  } else {
    // 6. External API call — ASYNC (network I/O via OS)
    //    Event loop: FREE while waiting
    const response = await fetch(`https://shipping-api/track/${orderId}`);
    shippingStatus = await response.json();  // ASYNC (reading response stream)

    // 7. Cache the result — ASYNC (network I/O to Redis)
    //    We DON'T await this — fire and forget
    redis.set(`shipping:${orderId}`, JSON.stringify(shippingStatus), 'EX', 300);
  }

  // 8. Combine and respond — SYNC
  const result = {
    ...order,
    shipping: shippingStatus,
  };

  // 9. Send response — SYNC write to buffer, OS sends async
  res.json(result);

  // 10. Log — SYNC (console.log is sync, writes to stdout)
  //     For production use async logger (pino) to avoid blocking
  console.log(`Order ${orderId} served in ${Date.now() - req.requestTime}ms`);
});
```

```
TIME BREAKDOWN for this request:

  Code execution (CPU on event loop):    ~0.5ms
  JWT verify (CPU):                      ~0.1ms
  DB query (I/O wait):                   ~20ms   ← event loop FREE
  Redis GET (I/O wait):                  ~1ms    ← event loop FREE
  External API (I/O wait):               ~150ms  ← event loop FREE (cache miss)
  Redis SET (fire-and-forget):           ~0ms    ← don't wait
  JSON serialization (CPU):              ~0.1ms
  ──────────────────────────────────────────────
  Total wall time:                       ~172ms
  Total EVENT LOOP BLOCKED:              ~0.7ms  ← Node served other requests
                                                    for the other 171.3ms
```

```
THIS is why Node.js scales:
  171.3ms of a 172ms request = idle event loop = available for other requests.
```

2.9 Complete Execution Model — Cheat Sheet

```
┌──────────────────────────────────────────────────────────────────────┐
│ WHAT GOES WHERE                                                      │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│ CALL STACK (main thread / event loop):                               │
│   ✅ Your JavaScript code                                            │
│   ✅ JSON.parse/stringify                                            │
│   ✅ Array/Object operations                                         │
│   ✅ jwt.verify (sync) — careful, RS256 is slow                      │
│   ✅ bcrypt.compareSync — ❌ NEVER use sync version                  │
│   ✅ console.log                                                     │
│   ✅ RegExp execution                                                │
│   ✅ Template rendering                                              │
│                                                                      │
│ OS ASYNC (epoll/kqueue — unlimited concurrency):                     │
│   ✅ TCP/UDP sockets (HTTP requests, DB connections, Redis)          │
│   ✅ dns.resolve() (uses c-ares library, not thread pool)            │
│   ✅ Pipes                                                           │
│   ✅ Signals                                                         │
│                                                                      │
│ LIBUV THREAD POOL (default 4 threads):                               │
│   ✅ fs.readFile, fs.writeFile, fs.stat (all fs.* async ops)         │
│   ✅ dns.lookup() (NOT dns.resolve — different mechanism)            │
│   ✅ crypto.pbkdf2, crypto.scrypt, crypto.randomBytes                │
│   ✅ zlib.gzip, zlib.deflate                                         │
│                                                                      │
│ WORKER THREADS (opt-in, separate V8 instance):                       │
│   ✅ CPU-heavy tasks you create explicitly                           │
│   ✅ Image processing, ML inference, heavy computation               │
│                                                                      │
├──────────────────────────────────────────────────────────────────────┤
│ QUEUE PRIORITIES (highest → lowest):                                 │
│                                                                      │
│   1. process.nextTick()          ← drains between every phase        │
│   2. Promise.then/catch/finally  ← microtask, after nextTick         │
│   3. setTimeout(fn, 0)           ← timers phase                      │
│   4. I/O callbacks               ← poll phase                        │
│   5. setImmediate(fn)            ← check phase                       │
│   6. close callbacks             ← close phase                       │
```

```
│                                                                    │
│   Inside I/O callback:                                             │
│     setImmediate > setTimeout (check is NEXT after poll)           │
│   In main module:                                                  │
│     setTimeout vs setImmediate → UNDEFINED ORDER                   │
│                                                                    │
├────────────────────────────────────────────────────────────────────┤
│ GOLDEN RULES:                                                      │
│                                                                    │
│   1. NEVER block the event loop with CPU work > 10ms               │
│   2. ALWAYS use async versions of fs, crypto, etc.                 │
│   3. NEVER use *Sync methods in server code                        │
│      (readFileSync, pbkdf2Sync, etc.)                              │
│   4. Monitor event loop lag — alert if > 100ms                     │
│   5. Offload CPU work to worker_threads                            │
│   6. Don't recurse with nextTick/Promises — use setImmediate       │
│   7. Set UV_THREADPOOL_SIZE if doing heavy fs/crypto               │
│   8. JSON.stringify on large objects CAN block — use streaming JSON │
│   9. RegExp with catastrophic backtracking CAN block — validate    │
│  10. The event loop is your heartbeat — keep it pumping            │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```